DOMinator

HTML & CSS Trainer for Interview Preparation

Team DevBand · RSS Stage 2 · 2026

🚀 Live Demo

Команда DevBand

Евгений Верёвкин

Tech Lead · Архитектура

Practice · Dashboard · LLM · Game Logic

Александра Потапова

Frontend · Backend

Library · Auth · Active Game · Supabase

Сергей Уразов

Frontend · UI

Modal · Layout · Landing · Auth Service

Ментор: iShvets · 5 командных встреч · ~180 PR за 7 недель

Что такое DOMinator?

Тренажёр HTML/CSS для подготовки к интервью.

- 📚 **Библиотека тем** — HTML & CSS топики по уровням сложности
- ❓ **Вопросы с вариантами** — генерируются LLM на лету
- 🎮 **Игровая механика** — очки, штрафы, super-game
- 💡 **Подсказки** — 50/50, звонок другу, объяснение
- 🔄 **Resume flow** — игра сохраняется, можно продолжить
- 🏆 **Ачивки** — Loser / Master / Guru по итогам

```
Пользовательский путь:

Landing
  └── Register / Login
        └── Library (выбор темы)
              └── Practice (игра)
                    ├── Super-Game (если ошибки)
                    └── Final Screen (ачивки)
                          └── Dashboard (история)
```

Scoring

+1 правильный · -1 неправильный
Super Game: all or nothing bonus

Проблема и цель

Проблема

Разработчики на входе в профессию не знают, **как готовиться к техническим интервью** по HTML и CSS. Документация есть везде — но нет инструмента для практики именно в формате интервью.

Цель

Создать интерактивный тренажёр с настоящими вопросами, уровнями сложности, подсказками и отслеживанием прогресса. Чтобы прийти на интервью — и **знать HTML/CSS**.

Tech Stack

Frontend

TypeScript + Vite
строгая типизация, быстрый dev-сервер

SCSS + CSS Custom Properties
design tokens, dark/light themes

Vitest + @testing-library/dom
123+ тестов, jsdom environment

ESLint + Prettier + Husky
GitHub Actions CI

Backend / Infra

Supabase
Auth · PostgreSQL · Edge Functions · RLS

LLM API (Edge Function)
генерация вопросов + кэширование в БД

Netlify
деплой продакшн-версии

Trello
командный трекер задач

Архитектура приложения

⚙️ App Core

Router + guards
Global store
Actions
Layout

📄 Pages

Landing
Auth
Library
Practice
Dashboard

🧩 Components

Game logic
Practice card
Hints
Modal
Final screen

🔌 Services

Auth service
Resume game
Sync to server
API layer

Ключевой принцип: страницы и компоненты не знают о Supabase — всё через сервисный слой. Состояние хранится в одном store, обновляется через actions, UI подписывается на изменения.

Глобальный State

Один объект — единый источник правды для всего приложения

👤 **user**

кто залогинен, токен сессии

🎮 **game**

тема, сложность, очки, раунд, подсказки, ошибки, режим (обычная / супер-игра)

🎨 **ui**

тема оформления, текущий маршрут, состояние меню

Где это хранится?

Данные	Где живут
Пользователь	`localStorage` + Supabase Auth
Активная игра	`localStorage` + таблица `active_games`
Настройки UI	`localStorage` (тема сохраняется между сессиями)

Почему это важно

Любой компонент — Header, игровая карточка, навигация — подписывается на store и получает обновления автоматически. Нет prop-drilling, нет рассинхрона.

Router: кто куда может попасть

🔓 Только для гостей

`/`	→ Landing
`/login`	→ Auth

🔐 Только для авторизованных

`/library`	→ Library
`/practice`	→ Practice
`/dashboard`	→ Dashboard

Как работают редиректы

👤 Гость открывает `/practice` → попадает на `/login`

✅ Авторизованный открывает `/` → попадает на `/library`

🔍 Неизвестный адрес → страница 404

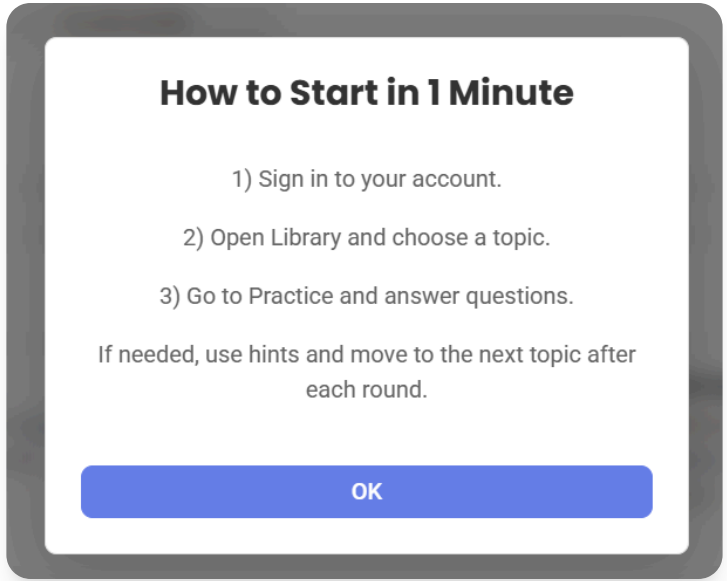
Написан с нуля на TypeScript без сторонних библиотек. При смене маршрута store автоматически обновляется — навигация подсвечивает активный пункт.

Модальное окно

Идея: модалка работает как диалог — вызываешь, ждёшь ответа, продолжаешь.

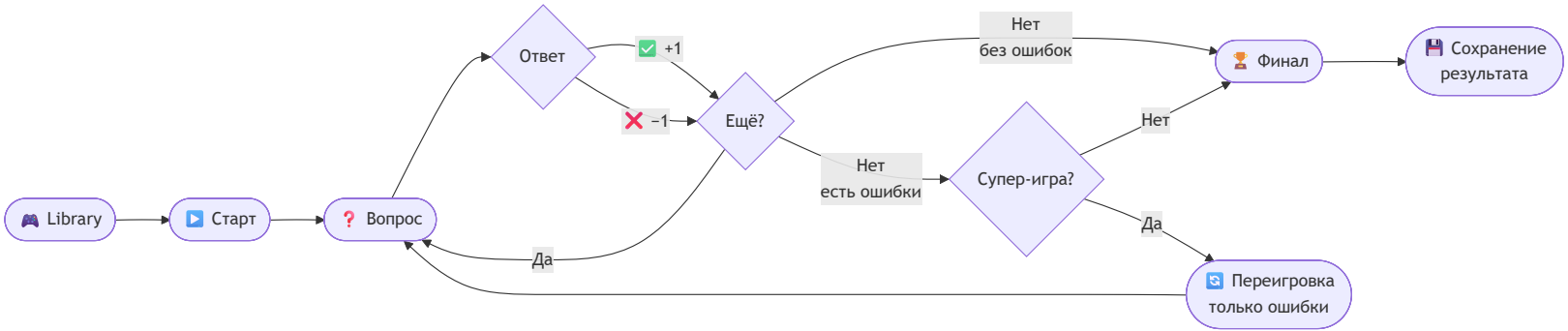
```
// Вызов выглядит как обычный вопрос
const result = await showModal({
  title: 'Продолжить игру?',
  confirmText: 'Продолжить',
  cancelText: 'Начать заново',
});

if (result.confirmed) {
  // пользователь нажал "Продолжить"
}
```

Никаких коллбэков — линейный читаемый код



Singleton · ESC / клик по фону закрывает · Promise API

Game Flow


Library
Старт
Вопрос
Ответ
+1
−1
Ещё?
Да
Нет
без ошибок
Нет
есть ошибки
Супер-игра?
Нет
Да
Финал
Сохранение
результата
Переигровка
только ошибки

Система подсказок (Hints)

50/50

Убирает два неправильных варианта ответа

`saveUsedHint('fifty')`

Call a Friend

Предлагает вероятный правильный ответ

`saveUsedHint('friend')`

I Don't Know

Открывает side-панель с объяснением вопроса

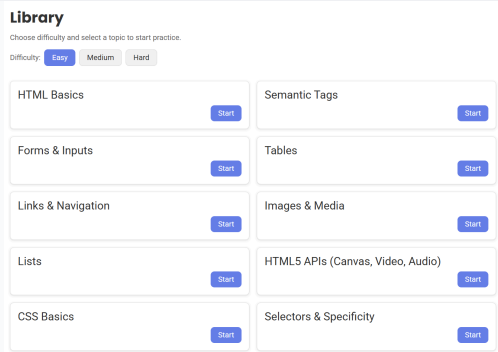
`saveUsedHint('explain')`

Persistence подсказок:

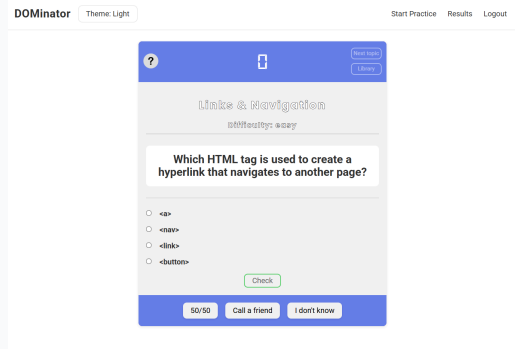
- Использованные хинты хранятся в `state.game.usedHints`
- Синхронизируются в `active_games` таблицу Supabase
- Кнопки дизейблятся через `usedHints[key] > 0`
- При refresh страницы состояние восстанавливается из store **до рендера**

Приложение в работе

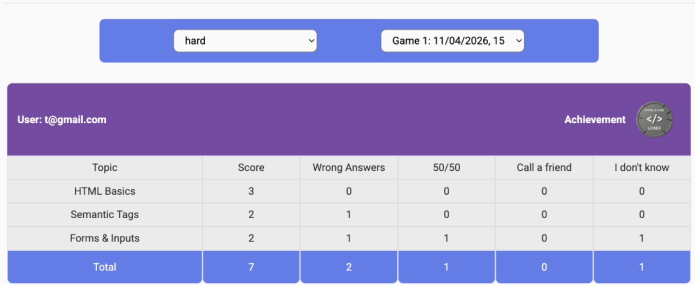
Library — выбор темы



Practice — игровой экран



Dashboard — история результатов



Topic	Score	Wrong Answers	50/50	Call a friend	I don't know
HTML Basics	3	0	0	0	0
Semantic Tags	2	1	0	0	0
Forms & Inputs	2	1	1	0	1
Total	7	2	1	0	1

Challenge 1: Вопросы из воздуха

Задача: откуда берутся вопросы по HTML/CSS?

❌ **Вариант — написать вручную**

Сотни вопросов по десяткам тем? Нереально.

✅ **Вариант — попросить LLM**

Supabase Edge Function получает тему и уровень сложности, запрашивает у LLM батч вопросов в нужном формате и сохраняет их в базу. Следующий запрос той же темы — из кэша, мгновенно.

Что было нетривиально:

🎯 Prompt нужно было отточить, чтобы LLM возвращал строгий JSON с правильным форматом вариантов ответа

📝 Каждый вопрос содержит поле *explanation* — именно оно показывается в подсказке «I Don't Know»

⚡ Первый запрос по теме ~1–2 сек. Повторный — из базы, мгновенно

Организация работы команды

Как мы строили процесс

🗓️ **5 регулярных встреч**
Старт → декомпозиция → распределение задач → ревью прогресса → финальный sprint

📋 **Trello для задач**
Backlog → In Progress → Done. Середине проекта навели порядок на доске — всё разъехалось

🔀 **PR-driven разработка**
feature/* → develop → main. Только через pull request, code review обязателен

Что было сложно

🔗 **Зависимые задачи**
Некоторые фичи было сложно делать параллельно — один блокировал другого. Учились договариваться заранее

🥱 **Устаревший код в develop**
Однажды участник не обновил ветку перед merge — старый код попал в общую. Дорогостоящая ошибка: пришлось делать rebase

✅ **Вывод**
`git pull origin develop` до начала любой задачи — стало правилом, а не рекомендацией

CI/CD: автоматика от коммита до деплоя

Pre-commit (Husky)

При каждом коммите автоматически запускается **lint-staged** — проверяет только изменённые файлы. Грязный код не попадёт в историю.

GitHub Actions (PR)

При открытии Pull Request запускается pipeline: **tsc** (проверка типов) → **eslint** → **vitest --run**. PR нельзя вмержить, пока всё не зелёное.

Netlify (деплой)

Merge в **main** — Netlify автоматически запускает сборку и публикует новую версию приложения. Zero-downtime deploy.

Итого: от коммита до продакшна — без ручных шагов. Если что-то сломалось — узнаешь ещё на этапе PR, а не после деплоя.

Ключевые технические решения

🛠️ Vanilla TypeScript, без фреймворка

SPA-роутер, state management, система подписок — всё написано с нуля. Показывает понимание основ, а не умение настраивать React.

🟢 Supabase как единая платформа

Auth, PostgreSQL, Edge Functions и хранилище — в одном сервисе. Для команды без выделенного бекенда это позволило сфокусироваться на продукте.

🤖 LLM для генерации контента

Вместо сотен вопросов вручную — Edge Function запрашивает LLM, сохраняет батч в БД. Повторный запрос — из кэша мгновенно. Контент масштабируется.

🪟 Promise-based модальное окно

`await showModal(...)` — любой диалог превращается в линейный код. Не нужно передавать колбэки через несколько уровней.

🔄 Тройной источник восстановления

Незавершённая игра ищется: в памяти → localStorage → Supabase. Пользователь никогда не теряет прогресс, даже при перезагрузке браузера.

🌓 Runtime смена темы

`data-theme` на `<html>` + CSS custom properties. Light/dark переключается без перезагрузки страницы, выбор сохраняется в localStorage.

Технические сложности

Supabase RLS и upsert

Создавать запись можно, обновить — нельзя. Оказалось, что upsert требует отдельных политик на INSERT и UPDATE. Заняло несколько часов отладки.

Игра завершалась дважды

Финальная логика сохранения срабатывала из двух мест одновременно. Починили явным await и единственной точкой ответственности.

Навигация до инициализации

При обновлении страницы приложение пыталось перейти куда-то ещё, хотя роутер ещё не был готов. Решили строгим порядком запуска.

Объяснение на чужой странице

Пока сервер отвечал (~2 сек), пользователь уходил на другую страницу — панель открывалась там. Добавили проверку: DOM-элемент ещё в документе?

Общий урок: асинхронный код без явного порядка и проверки контекста ведёт к непредсказуемым сайд-эффектам. Дисциплина **await** и проверки состояния обязательны.

Тестирование

12 тест-файлов · 123+ тестов

Что покрыто	
Глобальный store и actions	✅
Модальное окно	✅
Layout и навигация	✅
Landing (guest/authed CTA)	✅
Library + resume flow	✅
Авторизация	✅

🗄️ **localStorage в тестах не работает «как в браузере»**

В Vitest 4.x + jsdom он недоступен при инициализации модулей. Решение — мокировать сервис хранилища, а не сам API.

🟢 **Supabase в тестах не вызывается**

Тестируется логика приложения, а не доступность внешнего сервиса. Supabase подменяется моком на уровне api-слоя.

⚙️ **CI проверяет каждый PR**

GitHub Actions запускает `tsc + eslint + vitest --run`. Ни одна регрессия не уходит в develop незамеченной.

Система ачивок

Loser

Score ≤ 50%

Недостаточные знания.
Рекомендуется больше практики

Master

Score 51–85%

Уверенное знание с пространством для роста

Guru

Score 86–100%

Отличные знания и готовность к интервью

Ачивка отображается на final screen и **сохраняется в Supabase** — видна в истории игр на Dashboard

Сильные стороны проекта

Без фреймворка

Router, state, lifecycle — всё написано вручную на TypeScript. Понимание основ, а не магия фреймворка.

LLM-интеграция

Вопросы генерируются автоматически, кэшируются в БД. Контент масштабируется без ручного труда.

Устойчивость к сбоям

Игра восстанавливается из 3 источников. Обновил страницу, закрыл вкладку — прогресс не теряется.

Доступность (a11y)

Skip-link, aria-current, aria-pressed, keyboard navigation. Не как галочка, а как часть архитектуры.

123+ тестов

Каждый ключевой модуль покрыт. CI не пропустит регрессию в следующий релиз.

Code review культура

Ни одна задача не мержилась без ревью. Это улучшало архитектуру и передавало контекст всей команде.

Что получилось

Продукт

- ✅ Авторизация с сохранением сессии
- ✅ Библиотека тем с уровнями сложности
- ✅ Игровой экран с подсказками
- ✅ Супер-игра для проработки ошибок
- ✅ Восстановление незавершённых игр
- ✅ Dashboard с историей и ачивками
- ✅ Адаптив + dark/light тема

Команда

Первый опыт с Supabase

Auth, БД, Edge Functions, RLS — всё освоено в боевых условиях, не в туториале

Первый опыт с LLM в продакшне

Prompt engineering, обработка JSON-ответов, кэширование — реальная интеграция AI в приложение

Командная разработка от 0

Git workflow, code review, зависимые задачи, конфликты — всё из реальной командной работы

Выводы

🧭 Планирование спасает время

Задачи с зависимостями нужно выявлять до начала спринта, а не в середине — иначе один блокирует другого.

🔁 Итерации лучше идеального плана

Первая реализация resume flow была заменена трижды. Каждая итерация опиралась на опыт предыдущей.

📖 Документация — это артефакт команды

Дневники, meeting notes и CONTEXT_GLOBAL.md сэкономили часы при ревью и онбординге на новые задачи.

🤝 Code review — это обучение

Ревью не формальность. Именно там передавался контекст, находились баги и вырабатывались общие конвенции.

⚡ Async без дисциплины = хаос

Большинство наших сложных багов — это потерянный await или нарушенный порядок инициализации. Правила помогают.

🌿 Git — командный инструмент

Один несинхронизированный merge стоил команде нескольких часов. Договорённости важнее, чем скорость.

Итого

7	180+	123+	∞
недель разработки	pull requests	unit тестов	вопросов от LLM

🚀 **webis-2022-rs-tandem.netlify.app**

GitHub · RSS Stage 2 · Team DevBand · 2026